

AUDITORIA DE SEGURANCA

Relatório de Auditoria de Segurança - LIN (lin-open)

LIN / lin-open (kbelludoo/lin-open)

Data: 1 de setembro de 2026

Escopo	Arvore de trabalho do repositorio no commit fe4a10b (branch arena/01a05f60-lin-open, PR #55) e seu historico git completo (7 commits). Superficies auditadas: compiler/ (Zig 0.13.0, Stage0, 19 023 linhas), transpile/c/ (host C11: runtime de confianca lin_c/ + ferramentas tool/), src/*.lin (44 arquivos, incluindo o front-end escrito em LIN de #46), test/ (verificadores bash/python), .github/ (CI), benchmarks/verify_receipt.{html,js} (unica superficie de navegador do repo), docs/ e redteam/ (prior art).
Fora de escopo	Prova formal do parser/typechecker inteiro (19k linhas lidas por amostragem dirigida), binarios publicados (nenhum neste ambiente), e o comportamento do runtime OpenCL real (ja coberto por SECURITY_AUDIT.md + redteam/fake_ocl.c, referenciado como prior art).
Ambiente dos testes	Zig 0.13.0 real obtido via wheel ziglang==0.13.0 em venv isolado; Stage0 construido com zig build -Dgpu=false -Doptimize=ReleaseFast; host C11 (lin_c0, lin_bcl_run, lin_c_receipt) construido com cc. As duas implementacoes foram exercitadas nos mesmos vetores (ver 'Validacao nas duas versoes').

Nota metodológica - como cada categoria foi mapeada para esta stack

Projeto sem banco, sem ORM, sem framework web, sem sessao/usuario e sem multi-tenancy: as cinco categorias classicas foram traduzidas para os equivalentes desta stack.

1. BANCO SEM TRANCA (isolamento de tenant) -> ANCORA DE CONFIANCA / ESCOPO DA VERIFICACAO: em um verificador de integridade, o equivalente de 'filtrar por tenant' e resolver a identidade/confianca a partir de algo que NAO esta no objeto verificado. Auditado em: attest-verify, notary-verify, bundle/cert verify, gate-check/verify_gate_manifest.py, escopo do Merkle root vs arquivos reais.
2. PERMISSAO DEFINIDA NO NAVEGADOR -> DECISAO DE CONFIANCA DO LADO ERRADO: a pagina benchmarks/verify_receipt.html (e o .js) decidem 'verificado' no cliente; e o gate cuja verificacao de assinatura e opcional e re-atestando por quem adulterou.
3. IDOR (objeto por ID sem checar posse) -> REFERENCIA NAO VALIDADA: indices e comprimentos vindos de dados nao confiaveis (secoes/opcode/call/local da imagem LINBC1), nomes resolvidos por comprimento, e caminhos de arquivo escritos/executados sem validacao (src/lin.zig, .lin_ast_check.zig, LIN_ZIG).
4. CHAVES EXPOSTAS -> aplicada literalmente: seeds/keys Ed25519, defaults publicos que viram segredo real (authority.key, seed_prefix), ausencia de validacao de startup, tokens/acoes na CI, historico git, bundle do frontend.
5. XSS / INPUT SEM TRATAMENTO -> renderizacao nao-escapada na unica pagina HTML (nenhuma encontrada: so textContent), injecao no codigo Zig gerado a partir de input LIN, seguranca de memoria no parsing de bytes hostis em C11, e higiene de shell/python (shell=True, quoting, /tmp previsivel).

Ferramentas: leitura dirigida + grep padrão por categoria, e execução de PoCs (forja de roster, forja de receipt, adulteração de compilador + re-atestado, imagens LINBC1 mutadas, nomes com prefixo comum, nesting profundo) nos dois hosts. Nada foi reportado sem estar verificado no código ou medido; o que não se aplica esta dito como tal.

1. Resumo executivo

A auditoria encontrou **11 achados: 2 críticos, 2 altos, 3 medios, 2 baixos e 2 informativos**. O padrão dominante não é um bug de implementação e sim a **ausência de âncora de confiança**: os quatro mecanismos que o projeto apresenta como prova (atestado, certificado, quorum de testemunhas e receipt) verificam dados auto-declarados, e a página que 'verifica' o receipt decide isso no navegador. Em contrapartida, a máquina de baixo - o loader LINBC1 e a LinVM C11 - mostrou-se rigorosa e fail-closed em todos os testes hostis que rodamos, incluindo os dois verificadores independentes e o verificador Zig.

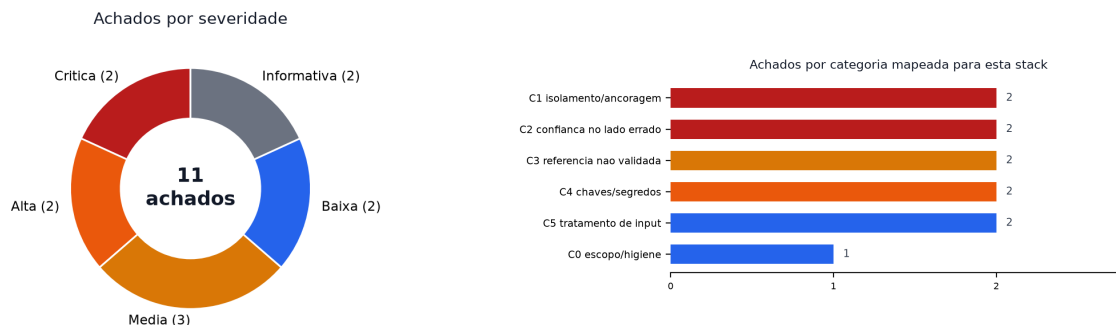


Tabela de achados (indice)

Sev.	ID	Achado	Onde	Cat
CRÍTICA	F-01	Verificadores criptograficos confiam na chave pública embutida no próprio documento verificado	compiler/lin.zig:8607-8621 (attest-verify); compiler/lin.zig:9806-9832 (verify-004); compiler/lin.zig:10392-10420 e 10623 (bundle-pack/bundle verify); compiler/lin.zig:11223 (ref externa)	C1
CRÍTICA	F-02	Quorum de testemunhas e auto-definido: roster de 1-de-1 com chave derivada de constante pública e aceito	compiler/lin.zig:7284-7338 (parse + politica de quorum)	C2
ALTA	F-03	Receipts 'cryptographically verified' são apenas autocoerencia de hash - e a decisão de 'verificado' acontece no navegador	compiler/lin.zig (receipt-verify)	C2
ALTA	F-04	Chave privada de autoridade gerada com permissão 0644 e nome de arquivo default no CWD	compiler/lin.zig:7643 e 8719 (default "authority.key")	C4
MÉDIA	F-05	O LIN Gate é auto-atestavel por default: sem roster, quem adultera o compilador também limpa o gate	Makefile:170-186 (gate/gate-attest)	C1
MÉDIA	F-06	Material de chave derivado de constante pública e sem validação de startup (defaults que viram segredo real)	compiler/lin.zig:12457 (seed_prefix default)	C4
MÉDIA	F-07	Escritas em caminho fixo dentro da arvore de trabalho e execução de binario escolhido por env/sonda de filesystem	compiler/lin.zig:7549 (lin rebuild escreve src/lin.zig)	C3
BAIXA	F-08	Documentação afirma gates/publicacao em CI que o workflow não contem (falsa garantia)	.github/workflows/ci.yml (92 linhas; nenhum job 'release' nem 'gate' nem 'compiler0-no-zig')	C5
BAIXA	F-09	CI sem bloco `permissions:` e com ação de terceiro referida por tag mutavel	.github/workflows/ci.yml:1-23 (sem `permissions:`)	C0
INFORMATIVA	F-10	Alegação tecnica falsa sobre o loader de confiança, duplicação da mesma lógica e comentario depreciativo no documento de fronteira	transpile/c/tool/lin_c0.c:156-165	C3

Sev.	ID	Achado	Onde	Cat
INFORMATIVA	F-11	Verificação de paridade/reportes com `steps=0` e status PASS; limites divergentes entre hosts	compiler/lin.zig (integrity/certificate)	C5

Validação nas duas versoes (Zig Stage0 x LinVM/C11)

Todos os vetores relevantes foram executados nas duas implementações. Concordancia nos goldens funcionais (vms_gate = value 1 em 8 511 500 passos em ambos; lex_gate, lex_scan_embedded e as rotas 'fonte' e 'imagem' idênticas) e **divergencia apenas nos limites**: o host C11 recusa C0_LIMIT_T00_MANY_TOKENS onde o Zig aceita 60 000 parenteses aninhados (fail-closed, correto). Os achados F-01, F-02, F-06 e F-07 são de rota Zig (o host C11 não implementa atestação/quorum); F-03 e F-10 aparecem nos dois; F-04/F-05/F-08/F-09 são de repositório/CI.

2. Pontos fortes verificados (o que esta protegido)

[OK] Loader LINBC1 fail-closed e com validação de referencias (o 'IDOR' de bytecode)

transpile/c/lin_c/lin_linbc1.c:114-170 valida name_idx, nparams<=nlocals, nlocals<=64, n_arrs<=8, alen em 1..256, duplicata de descritor, jump/call/local fora de alcance e contagem total de instruções; 184-189 confere o self-hash `linbc1:img:` nos 32 bytes finais. PoC: quatro imagens mutadas de 1 byte (contagem de strings, corpo, cauda, campo de tamanho) => todas `REJECTED` (`linbc1.selfhash` / `linbc1.flags`), sem crash, nos dois hosts (`lin\_bc1\_run` e `lin\_c0 run`).

[OK] Nenhuma API de risco no host C11

grep de `strncpy|strcat|sprintf|gets|alloca|system|popen|execv|scanf` em transpile/c/ => zero ocorrencias fora de comentarios; o runtime e zero-allocation (arena fixa) e o loader não alocada: tudo em armazenamento do chamador, com os big-objects declarados `static` (transpile/c/tool/lin_bc1_run.c:36-37 e tool/lin_c0.c:171-172) - não estouram pilha.

[OK] Sem injeção no código Zig gerado por `lin test`

compiler/lin.zig:6989-7001 injeta nomes de função no texto Zig compilado e executado, porem a lista vem de `lin\_check`/'read_ident', que corta no primeiro caractere fora de identificador. PoC medido (Zig e C11 concordam): `!test\_a`; `vira` `test\_a`, `test.a` vira `test`, `test\_x`{'` vira `test_x` - aspas/parenteses/unicode nunca chegam ao gerador.

[OK] Verificadores auxiliares bem escritos (python/bash)

`subprocess.run` sempre com lista de argumentos (sem `shell=True`) em test/verify_gate_manifest.py:86-91, test/verify_compiler0_manifest.py:72 e examples/*.py; bash com `set -euo pipefail` e `mktemp -d` + `trap` (test/verify_c0_selfhost.sh:19-22, test/verify_c0.sh, verify_linvm0.sh:21-27 falha se o binario não existe em vez de passar).

[OK] Página do verificador não tem XSS

benchmarks/verify_receipt.html escreve tudo com `textContent` (linhas 121-140) e valida o digest com regex `/^[0-9a-fA-F]{64}\$/` ; `BigInt` falha fechado para campos não numericos. Nenhum uso de `innerHTML`, `insertAdjacentHTML`, `eval`, `new Function` no repo (grep em *.html/*.js/*.mjs => 0). O risco real da página e outro (F-03): o que ela verifica.

[OK] Modo assinado do gate e fail-closed quando acionado; gate-keygen protege a chave

`gate-check --roster` sobre manifesto não assinado => 'GATE BLOCKED - attestation signature not verified: 0 valid signature(s), 1 required' (medido). `gate-keygen` cria `lin\_gate\_key.seed` com `.mode = 0o600` (medido: stat = 600), e o roster tem politica de maioria (`\*2 <= n` rejeitado).

[OK] GPU não finge PASS sem OpenCL real

`lin\_native gpu-verify examples/map\_kernels.lin` no build `-Dgpu=false` => 'no OpenCL platforms found' e saída não-zero (medido). O PoC histórico de runtime OpenCL falso (redteam/fake_ocl.c + SECURITY_AUDIT.md §1) permanece documentado e a conclusão 'paridade e conformidade, não prova criptografica' continua valida - mas não ha mais PASS falso por default.

[OK] Histórico e arvore livres de segredos commitados

Varredura completa: `git log --all -p` (7 commits) para padrões GitHub PAT/AWS/xox/PEM PRIVATE KEY/'SECRET_KEY='/'PASSWORD=' => zero ocorrencias; nenhum `.env`, `\*.key`, `\*.pem` ou seed em `git ls-files`; .gitignore e allowlist (`\*` ignorado por default), entao chaves geradas em `authority.key`/'*.seed`/'witness\_roster.rule1` caem fora do commit. O repo usa `secrets.\*` em nenhum ponto e não tem `pull\_request\_target`.

[OK] Auto-digests recomputados por verificador independente

test/verify_compiler0_manifest.py:44-52 recalcula em Python o hash do formato (`linbc1:img:` sobre os bytes do arquivo) em vez de confiar no emissor, e falha com rc=1 quando o pin e alterado (PoC negativo medido: rc=1; golden trocado no gate => FAIL). As imagens são reproduzíveis byte a byte (medido 3x).

3. Riscos centrais

[!] As 'provas' do projeto não tem âncora de confiança

Attestados, certificados, rosters de testemunhas e receipts são verificados contra dados que o próprio documento traz (F-01, F-02, F-03). Isso não é um bug pontual: é a ausência de um mecanismo de ancoragem (roster pinado, política mínima, re-execução). E o equivalente, nesta stack, de 'query sem filtro de tenant' - a verificação não sabe a quem pertence a confiança.

[!] Controles que só valem se o adversário não puder escrever no mesmo lugar

O gate lê a raiz de um manifesto versionado que o próprio PR pode regravar (F-05); o roster vem do CWD (F-02); a chave de autoridade vive no CWD com permissão ampla (F-04); o compilador externo vem de uma variável de ambiente (F-07).

[!] Rótulos mais fortes do que a mecânica

'cryptographically verified', 'ATTESTATION VALID', 'PROVENANCE ... Verified authentic', 'a gate that blocks a pull request', 'CI pública assets' (F-03, F-05, F-08). Em projeto guiado pela regra R5 (sem overclaim), esses termos são parte da superfície de segurança.

[!] Escritas e execuções com caminho implícito

F-07: ``src/lin.zig`` sobrescrito, ``.lin_ast_check.zig`` no CWD, binário por ``.LIN_ZIG``, sonda ``.home/k/...`` - comportamento que depende do ambiente em rotas de verificação.

[!] Porta sem-Zig com garantias menores do que a porta com Zig

A rota ``.python3 test/verify_gate_manifest.py`` (usada pelo job ``.compiler0-no-zig`` proposto) não verifica assinatura nem implementa quorum; e não existe verificador de receipt em C11 (``.tool/lin_c_receipt.c`` só emite) (F-03, F-05).

4. Achados detalhados por categoria

Cada achado traz severidade, arquivo:linha, trecho de evidência, o motivo pelo qual é explorável, condições de explorabilidade, impacto e correção sugerida. As linhas foram confirmadas no código do commit auditado.

4.1 C1 - BANCO SEM TRANCA (isolamento/ancoragem)

CRÍTICA	F-01 - Verificadores criptograficos confiam na chave pública embutida no próprio documento verificado
Arquivo:linha	compiler/lin.zig:8607-8621 (attest-verify); compiler/lin.zig:9806-9832 (verify-004); compiler/lin.zig:10392-10420 e 10623 (bundle-pack/bundle verify); compiler/lin.zig:11223 (ref externa)

Evidência

```
// attest-verify - compiler/lin.zig:8607-8621
// 7. Decode Key & Signature DIRECTLY from Document & Cryptographically Verify
if (doc.pubkey_hex.len != 64 or doc.signature_hex.len != 128) {
    return error.InvalidKeyOrSignatureLength;
}
var pubkey_raw: [32]u8 = undefined;
_ = try std.fmt.hexToBytes(&pubkey_raw, doc.pubkey_hex);
...
const pubkey = try std.crypto.sign.Ed25519.PublicKey.fromBytes(pubkey_raw);
try sig.verify(canonical_msg, pubkey); // <- a chave vem do documento assinado
```

Por que é explorável	A assinatura Ed25519 é válida contra a chave pública que o próprio documento declara. Qualquer pessoa gera um par de chaves, assina as próprias alegações e obtém um documento que passa em todas as rotas de verificação. Não há roster pinado, nenhuma âncora externa e nenhuma checagem de revogação nessas rotas. O resultado impresso é 'ATTESTATION VALID' / 'PROVENANCE ... Verified authentic git lineage'.
Condições de explorabilidade	Sem condição: qualquer `rule` produzido com `attest-issue --key <arquivo meu>` (ou com chave própria + edição dos campos) é aceito. O `nversion`/`cert` seguem o mesmo padrão de ancoragem.
Impacto	Provas de execução/provenance deixam de ser provas. Um agente que queira publicar um resultado adulterado pode emitir atestado 'valido' e o verificador de terceiros confirma. Destroi a premissa central do projeto (receipt/attestation verificável).
Correção sugerida	Resolver a chave contra uma âncora fora do documento: roster/allowlist de authorities pinado em arquivo versionado e assinado (ou policy de transparência tipo Sigstore), e recusar quando `pubkey_hex` não pertence ao conjunto. Checar revogação/epoch na mesma rota. O roster de notários deve chegar por canal fora de faixa (out-of-band), não do diretório do artefato.

CrITÉrios de aceite

- [] `attest-verify` de um documento assinado por chave fora do roster pinado sai BLOCKED com código nomeado.
- [] O roster usado na verificação não é lido do CWD nem do mesmo diretório do documento.
- [] Teste adversário: `attest-issue --key <chave nova>` + `attest-verify` => REJEITADO (hoje PASSA).
- [] Documentação deixa de dizer 'authenticated' sem âncora de confiança explícita.

MÉDIA	F-05 - O LIN Gate é auto-atestável por default: sem roster, quem adultera o compilador também limpa o gate
Arquivo:linha	Makefile:170-186 (gate/gate-attest, GATE_ROSTER opcional), test/verify_gate_manifest.py:137-142 e 164-165 (sem suporte a roster/assinatura), compiler/lin.zig:15678-15692 (claim 'a gate that blocks a pull request - AI-authored or not'), .github/workflows/ci.yml (nenhum job chama o gate)

Evidência

```
# Makefile:176-186
gate: build-cpu
ifeq ($(GATE_ROSTER),)
@$(BIN) gate-check
else
@$(BIN) gate-check --roster $(GATE_ROSTER)
endif
# test/verify_gate_manifest.py:165
print(" manifesto escrito: %s (attested_by=%s - NAO assinado)" % (manifest, args.attested_by))
```

Por que é explorável	O gate detecta adulteração, mas não prova quem a re-atestou: `python3 test/verify_gate_manifest.py --attest` regravava a raiz e o `gate-check` volta a dizer OPEN. PoC neste ambiente: acrescentei uma linha em `compiler/lin.zig` => `error: GateBlocked` e 'GATE BLOCKED' no python; rodei o `--attest` => 'GATE OPEN - a raiz do disco coincide com a atestada'. O modo assinado existe e é fail-closed (`gate-check --roster` sobre manifesto não assinado => 'GATE BLOCKED - attestation signature not verified', medido), porém e opcional, o roster é lido de caminho arbitrário (logo editável no mesmo PR) e a porta sem-Zig nem implementa assinatura.
Condições de explorabilidade	Explorabilidade real = 'quem pode abrir PR contra a master': não precisa de segredo, apenas do direito de editar o arquivo de manifesto (o mesmo direito já dá para editar o compilador). A mitigação depende de branch protection + revisão humana, não do gate.
Impacto	Controle de integridade de supply chain descrito como bloqueante e, na prática, advisory; em particular o job `compiler0-no-zig` proposto (docs/CI_COMPILER0_NO_ZIG_JOB_PATCH.rule!) usa apenas a porta python não-assinada.
Correção sugerida	Tornar `--roster` obrigatório no `make gate` (com roster pinado fora da árvore do PR ou assinado por maintainers), portar a checagem de assinatura para `test/verify_gate_manifest.py`, e registrar o gate num job de CI (bloqueando merge).

Critérios de aceite

- [] `make gate` sem GATE_ROSTER falha (não é mais modo padrão).
- [] `python3 test/verify\_gate\_manifest.py --roster <R>` reproduz o veredito assinado do Zig.
- [] O job de CI executa gate + roster e o merge exige check verde.
- [] Adulterar compiler/lin.zig + retestar sem a chave do maintainer => CI vermelho (prova de PoC).

4.2 C2 - PERMISSAO/CONFIANCA DECIDIDA NO LADO ERRADO

CRÍTICA	F-02 - Quorum de testemunhas e auto-definido: roster de 1-de-1 com chave derivada de constante pública e aceite
Arquivo:linha	compiler/lin.zig:7284-7338 (parse + politica de quorum), 7363-7380 (verifyOne), 12455-12615 (notary-sign, seed default), 12643-12700 (notary-verify le witness_roster.rule!) do CWD)

Evidência

```
// compiler/lin.zig:7284-7292 - tudo vem do MESMO arquivo que se verifica
pub fn parse(alloc, bytes) !Parsed {
    if (!std.mem.startsWith(u8, bytes, "@RULEL:LIN_WITNESS_ROSTER:1.0.0")) ...
        .quorum_threshold = findIntField(bytes, "quorum_threshold") orelse ...,
}
// compiler/lin.zig:7337-7341 - politica aceita 1-de-1
if (parsed.head.quorum_threshold == 0) return error.QuorumThresholdZero;
if (parsed.head.quorum_threshold > n) return error.QuorumThresholdExceedsRoster;
if (parsed.head.quorum_threshold * 2 <= n) return error.QuorumThresholdNotMajority;
// compiler/lin.zig:12457 - material de chave publico por default
var seed_prefix: []const u8 = "lin-transparency-selftest";
```

Por que é explorável

O arquivo de roster contém simultaneamente as identidades (pubkeys), as assinaturas e o limiar de quorum. Como `quorum\_threshold=1` com `N=1` passa na checagem de maioria e as chaves podem ser regeneradas de um prefixo público, o verificador aceita o checkpoint que um atacante quiser. PoC executado neste ambiente: `lin notary-sign -o forged.rule! --witnesses 1 --threshold 1 --tree-size 999999 --epoch 7 --state-root sha256:aaaa...` seguido de `lin notary-verify` => '[QUORUM SATISFIED]' e 'TRANSPARENCY CHECKPOINT VERIFIED', com receipt escrito.

Condições de explorabilidade	Explorável por qualquer pessoa que consiga colocar `witness_roster.rule1` no diretório de execução (o caminho é fixo e relativo ao CWD: compiler/lin.zig:12643) - por exemplo no mesmo tree de um PR. `notary-verify <arquivo>` ignora o argumento posicional e lê o default, então nem o operador percebe a troca.
Impacto	Log de transparência forjável de ponta a ponta: árvore, epoch e state root arbitrários 'notarizados'. Todo consumidor do receipt `notarization_status=QUORUM_VERIFIED_FROM_ROSTER` é enganado.
Correção sugerida	Pin set de pubkeys de notários fora do espaço do artefato (arquivo assinado / config de deploy), exigir `N >= 3` e `M > 2N/3`, recusar rosters com testemunhas novas e exigir que `--roster` seja obrigatório (sem modo 'sem testemunhas'). Rejeitar argumentos posicionais desconhecidos em rotas de verificação.

Critérios de aceite

- [] `notary-verify` falha se `quorum\_threshold`/`N` não baterem com a política pinada ($N \geq 3$).
- [] Um roster com testemunha desconhecida => 'QUORUM NOT SATISFIED' + exit != 0.
- [] `notary-verify <arquivo>` usa o arquivo informado ou erro de uso (não lê o CWD as escondidas).
- [] Nenhum default de seed prefix em rota que gera chaves; `--seed-prefix` passa a ser obrigatório.

ALTA	F-03 - Receipts 'cryptographically verified' são apenas autocoerência de hash - e a decisão de 'verificado' acontece no navegador
Arquivo:linha	compiler/lin.zig (receipt-verify), benchmarks/verify_receipt.js:1-20 e 96-131, benchmarks/verify_receipt.html:19-27 e 100-140, tool/lin_c_receipt.c (so emissor, sem verificador na porta C11)

Evidência

<pre>// benchmarks/verify_receipt.html:100-140 (e o mesmo em verify_receipt.js) const computed = await sha256(leaf); // leaf = campos do proprio receipt let ok = computed === claimed; if (source) { ... } // checagem do artifact e OPCIONAL out.textContent = ... + (ok ? "[PASS] Receipt validated by an independent open-source verifier" : "[FAIL] Receipt forged or tampered!");</pre>	
Por que é explorável	input, output, steps e sp_at_ret vem do receipt; o verificador só recomputa SHA-256(leaf) e compara. Não há chave, assinatura, nem re-execução. PoC neste ambiente: receipt forjado com `output=12345` (a função é `x*x`, entrada 9, saída correta 81) => `node verify_receipt.js` imprime PASS e `lin receipt-verify --receipt` imprime 'PASS: Compute Receipt ... cryptographically verified!'.
Condições de explorabilidade	Sem condição: basta saber a fórmula aberta do formato, documentada na própria página ('Open format: merkle_root = SHA-256(...)').
Impacto	Falsa sensação de prova. Um verifier terceirizado (a página é distribuída como 'independent') aprova qualquer resultado. O rótulo 'cryptographically verified' e overclaim: o receipt honestamente se declara `verification_level=0` / `INTEGRITY_RECEIPT_LOCAL`, mas a saída do verificador não preserva esse nível.
Correção sugerida	Renomear o veredicto para o que ele é ('autoconsistente / nível 0') e imprimir o nível declarado no receipt; para virar prova, vincular o receipt à execução: re-executar na LinVM (`lin_c0 vm` / `lin vm`) e comparar value+steps+sp_at_ret, ou então exigir assinatura de um emissor pinado (ver F-01). Manter a checagem do `artifact` obrigatória quando houver fonte.

Critérios de aceite

- [] A página/CLI exibem `verification\_level` e a frase do veredicto não usa 'cryptographic' para nível 0.
- [] Receipt com output alterado e `--source` informado => FAIL (hoje o source check é opcional).
- [] Um verificador de nível 1+ (re-execução na VM) existe e falha para receipt forjado.
- [] Nenhum caminho de CI apresenta nível 0 como prova de execução.

4.3 C3 - REFERENCIA NAO VALIDADA (IDOR analógico)

MÉDIA	F-07 - Escritas em caminho fixo dentro da árvore de trabalho e execução de binário escolhido por env/sonda de filesystem
--------------	---

Arquivo:linha compiler/lin.zig:7549 (lin rebuild escreve src/lin.zig), 7129-7137 (pick_target escreve .lin_ast_check.zig no CWD e prefere /home/k/.local/bin/zig se existir), 7018-7027 (LIN_ZIG decide o `zig run` do código gerado por `lin test`)

Evidência

```
// compiler/lin.zig:7549 - write em caminho fixo do source tree
const tf = try std.fs.cwd().createFile("src/lin.zig", .{});
// compiler/lin.zig:7129-7137 - alvo escolhido por presença de arquivo no HOME de um dev
const tmp_file = ".lin_ast_check.zig";
if (std.fs.cwd().createFile(tmp_file, .{})) |f| { ... }
if (std.fs.openFileAbsolute("/home/k/.local/bin/zig", .{})) |zf| { zb = "/home/k/.local/bin/zig"; }
// compiler/lin.zig:7018-7027
var zb: []const u8 = "zig";
if (std.process.getEnvVarOwned(LIA_ALLOC, "LIN_ZIG")) |p| { zb = p; }
return std.process.Child.run(.{ .argv = &.{ zb, "run", "-0", "ReleaseFast", tmp_path }, ... });
```

Por que é explorável	Tres problemas no mesmo ponto: (i) `rebuild` sobrescreve `src/lin.zig` sem checagem de symlink nem confirmação - arquivo pre-criado como link vira escrita fora da arvore; (ii) `pick_target` faz o resultado da verificação depender de um caminho absoluto de um desenvolvedor (`/home/k/...`), entao 'zig' vs 'js' muda de maquina para maquina num projeto que vende reprodutibilidade bit a bit; (iii) o binario do compilador usado para compilar e EXECUTAR o código gerado vem de `LIN_ZIG`, sem validação alguma.
Condições de explorabilidade	Requer a vitima rodar `lin rebuild`/`lin test` num diretorio controlado por terceiro (compartilhado, CI com checkout de PR, workspace de review) ou com ambiente em que o atacante seta variaveis.
Impacto	Escrita arbitraria/quebra de arquivo-fonte; resultado de verificação não reproduzível; execução de binario arbitrario com o código gerado a partir do input (RCE em fluxo de review/CI).
Correção sugerida	Escrever em caminho explicito via flag (sem default no source tree), abrir com `O_NOFOLLOW`/verificar symlink, e exigir `--zig <caminho>` (ou resolver so pelo PATH, com hash esperado) para o compilador externo; substituir a sonda `/home/k/...` por config explicita.

Critérios de aceite

- [] `lin rebuild` recusa escrever fora do diretorio informado e não toca em caminho fixo.
- [] Nenhum caminho absoluto de usuario aparece no código-fonte (grep `/home/` em compiler/*.zig => 0).
- [] `lin test` com `LIN\_ZIG=/bin/false` falha com erro nomeado, sem executar o que foi gerado.
- [] O alvo escolhido (zig/js) sai impresso com a justificativa e e identico em duas maquinas limpas.

INFORMATIVA

F-10 - Alegação tecnica falsa sobre o loader de confiança, duplicação da mesma lógica e comentario depreciativo no documento de fronteira

Arquivo:linha transpile/c/tool/lin_c0.c:156-165, docs/V2_LINVM_AS_COMPILER0_NOZIG.rule1 (secao 7), compiler0_manifest.rule1, HISTORY.rule1 (entrada C0-no-Zig cont.)

Evidência

```
/* transpile/c/tool/lin_c0.c:156-165
 * Busca exata pelo nome usando name_len[] do loader: o pool LINBCL nao e
 * NUL-terminado, entao `strlen` (como faz lin_bcl_find_fn) so e seguro no
 * ultimo nome da secao. Aqui e comparacao por comprimento - correta sempre. */
static int bcl_find_fn_strict(const LinBclVm *m, const char *name) { ... }
// transpile/c/lin_c/lin_linbcl.c:195-202 (o loader JA compara por comprimento)
int lin_bcl_find_fn(const LinBclVm *m, const char *name) {
    size_t want = strlen(name);
    for (size_t i = 0; i < m->mod.fns_len; i++)
        if ((size_t)m->name_len[i] == want && memcmp(m->mod.fns[i].name, name, want) == 0)
            return (int)i;
    return -1; }
```

Por que é explorável	O `strlen` do loader é aplicado no nome solicitado (argv), nunca na pool; a comparação já usa `name_len[i]`. Medido: `lin_bc1_run` e `lin_c0_run` resolvem nomes em qualquer posição (`lex_gate`, `lex_scan_embedded`, `lex_count_embedded` = 1 / 1371904279658369433 / 20) e recusam nome inexistente. O par de nomes com prefixo comum (`gate`, `gate_x`) resolve corretamente pelas duas rotas. Logo: não ha vulnerabilidade, mas a alegação publicada (doc §7, HISTORY e comentario) afirma que o TCB tem um furo e duplica a função do loader 'por segurança'.
Condições de explorabilidade	Não explorável. O custo e de confiança/revisao: quem audita confia menos no loader do que deveria e mantem dois codigos identicos.
Impacto	Documentação de fronteira incorreta + lógica redundante a `tool/` (manutencao).
Correção sugerida	Corrigir comentario e docs (remover a alegação), e ou chamar `lin_bc1_find_fn` diretamente ou justificar a copia com o motivo real (independencia de host), não com um defeito inexistente.

Critérios de aceite

- [] grep 'so e seguro no último nome' não retorna nada no repo.
- [] Seção 7 do doc descreve `bc1\_find\_fn\_strict` como comparação por comprimento equivalente a do loader.
- [] Um teste cobre resolução de nomes em primeira/meia/ultima posição (rota fonte e rota imagem).

4.4 C4 - CHAVES E SEGREDO EMBUTIDO

ALTA	F-04 - Chave privada de autoridade gerada com permissão 0644 e nome de arquivo default no CWD
Arquivo:linha	compiler/lin.zig:7643 e 8719 (default "authority.key"), 8038-8044 e 9170-9176 (geracao + createFile sem modo), contraste correto em compiler/lin.zig:15761 (gate-keygen usa .mode = 0o600)

Evidência

```
// compiler/lin.zig:7643 - default relativo ao CWD
var key_path: []const u8 = "authority.key";
// compiler/lin.zig:8038-8044 - gera e escreve sem fixar modo
std.crypto.random.bytes(&authority;_seed);
_ = try std.fmt.bufPrint(&key;_hex_buf, "{s}", .{std.fmt.fmtSliceHexLower(&authority;_seed)});
const kfile = try std.fs.cwd().createFile(key_path, .{});
try kfile.writeAll(&key;_hex_buf);
// compiler/lin.zig:15761 - a outra rota FAZ o certo
const kf = try std.fs.cwd().createFile(key_out, .{ .truncate = true, .mode = 0o600 });
```

Por que é explorável	`createFile(path, .{})` usa 0o666 & ~umask => 0644 com a umask padrão: o seed privado de 256 bits fica legível por qualquer usuario da maquina e por qualquer job/artefato que cole o diretorio (upload-artifact, `git add -A`, tarball de debug). O próprio repo já sabe fazer certo (`gate-keygen` usa 0o600, medido: `lin_gate_key.seed` saiu 600), entao e uma inconsistencia na rota de attest - não falta mecânica, falta aplica-la ali.
Condições de explorabilidade	Local: qualquer processo do mesmo host le o seed. Em CI: um `upload-artifact` com caminho amplo (`**`) pública o arquivo; o .gitignore do repo e allowlist, entao `authority.key` não seria commitado - mas um `git add -f` ou um bundle da arvore expoe.
Impacto	Roubo da identidade de assinatura de atestados: o atacante passa a emitir atestados validos (com F-01, sem nem precisar do verificador da vitima).
Correção sugerida	Usar `.mode = 0o600` em todo createFile de material sensível (8041, 9173), recusar default `authority.key` se o arquivo já existir e não for modo 0600 (validação de startup), e imprimir aviso quando o caminho estiver dentro de uma arvore de trabalho.

Critérios de aceite

- [] Apos `attest-issue`, `stat -c %a authority.key` == 600 (hje não roda porque o arquivo existe antes de mim).
- [] Rodar com `--key <arquivo existente 0644>` => erro nomeado 'key file is group/other readable'.
- [] grep por `createFile(` em rotas de chave não deixa nenhum call sem `.mode`.

[] Teste de CI: `upload-artifact` com `path: .` falha se houver chave no tree.

MÉDIA

F-06 - Material de chave derivado de constante pública e sem validação de startup (defaults que viram segredo real)

Arquivo:linha compiler/lin.zig:12457 (seed_prefix default), 7383-7395 (deriveKeypair), 12591-12601 (id da testemunha contem o prefixo), 12613-12615 (apenas um WARNING impresso)

Evidência

```
// compiler/lin.zig:12457 + 7386-7395
var seed_prefix: []const u8 = "lin-transparency-selftest";
pub fn deriveKeypair(seed_prefix: []const u8, index: usize) !KeyPair {
    h.update("lin:transparency:witness-key:v1:"); h.update(seed_prefix); ...
    return Ed25519.KeyPair.create(seed); // deterministico a partir do prefixo
}
```

Por que é explorável	O comentario no código é honesto ('NOT a root of trust'), mas o default esta vivo em rota de linha de comando que gera chaves e pública roster; o ID da testemunha ainda carrega o prefixo, facilitando reuso. Não ha flag que recuse produzir um roster 'de selftest' num contexto de release, nem checagem de que o prefixo veio de fora.
Condições de explorabilidade	Basta rodar `lin notary-sign` sem `--seed-prefix` (default) e usar o resultado como prova - e o que F-02 demonstra.
Impacto	Chaves de testemunha colecionadas por qualquer pessoa => quorum sem valor (multiplica F-02).
Correção sugerida	Exigir `--seed-prefix`/`--key` explicito (sem default) e, em modo release, importar pubkeys de notarios de arquivo assinado; marcar o roster gerado com `selftest=true` e fazer o verificador recusar esse flag por default.

Crítérios de aceite

- [] `notary-sign` sem seed prefix => erro de uso (hoje assume default).
- [] Roster autoassinado carrega `selftest=true` e `notary-verify` o recusa a menos de `--allow-selftest`.
- [] README/docs de deploy não mostram mais o comando sem `--seed-prefix`.

4.5 C5 - TRATAMENTO DE INPUT (XSS/injecao)**BAIXA**

F-08 - Documentação afirma gates/publicacao em CI que o workflow não contem (falsa garantia)

Arquivo:linha .github/workflows/ci.yml (92 linhas; nenhum job 'release' nem 'gate' nem 'compiler0-no-zig'), docs/LIN_SEM_ZIG_CAMINHOS.rule1:24-31, docs/CI_RELEASE_E_V1_GATES_PATCH.rule1, docs/CI_COMPILER0_NO_ZIG_JOB_PATCH.rule1, HISTORY.rule1:78 (nucleus.lock.json inexistente)

Evidência

```
docs/LIN_SEM_ZIG_CAMINHOS.rule1: "Este PR fecha essa divergencia: a CI agora sobe os
artifacts lin_native-cpu/lin_native-gpu em toda execucao e publica os dois como assets de
Release ao empurrar tag `v*` (job `release`, --verify-tag)."
$ grep -c "release" .github/workflows/ci.yml -> 0
$ ls nucleus.lock.json -> No such file or directory
```

Por que é explorável	O texto diz que a CI pública assets verificaveis; o workflow em arvore não tem o job (o patch esta em `docs/CI_*_PATCH.rule1`, aguardando aplicação por falta de escopo `workflows`). Quem seguir o doc para 'baixar e verificar' não encontra o asset nem a verificação. O arquivo `nucleus.lock.json`, citado como trava de integridade do verificador, não existe.
Condições de explorabilidade	Sem condição tecnica; e leitura de documentação. Materializa-se quando alguém confia na promessa em vez do manifesto.
Impacto	Expectativa de seguranga falsa sobre a cadeia de suprimento do binario; usuarios instalam asset inexistente/nao-verificado.
Correção sugerida	Deixar o status explicito nos docs ('patch pronto, NAO aplicado' já esta em LIN_SEM_ZIG_CAMINHOS:linhas de faixa 1, mas a frase do release não) ou aplicar os jobs; remover a referencia a `nucleus.lock.json` ou reintroduzi-la.

Critérios de aceite

- [] Nenhum documento afirma que a CI pública assets enquanto o job não existir.
- [] Referencias a arquivos de trava apontam para arquivos presentes no tree.
- [] README/docs listam quais gates rodam em CI hoje (nenhum) e quais rodam so locais.

INFORMATIVA

F-11 - Verificação de paridade/reportes com `steps=0` e status PASS; limites divergentes entre hosts

Arquivo:linha compiler/lin.zig (integrity/certificate), transpile/c/tool/lin_c0_front.c (C0_LIMIT_*), SECURITY_AUDIT.md (prior art)

Evidência

```
$ ./zig-out/bin/lin_native integrity | tail -7
.compiler_sha256="16272e2f..." .corpus_sha256="28b64d74..."
.targets=20 .confirmed=20 .refuted=0 .steps=0 .status="PASS"
$ lin_c0 vm -> .fatal{ reason="C0_LIMIT_TOO_MANY_TOKENS" }
$ lin_native check -> OK (parser iterativo)
```

Por que é explorável Dois pontos de higiene, não falhas: (i) `integrity` declara PASS sem executar nenhum passo (e conformidade de hashes, como SECURITY_AUDIT.md já concluiu - o status deveria dizer isso); (ii) o host C11 é mais estrito que o Stage0 em tokens, então um fonte válido no Zig pode ser recusado na LinVM (fail-closed e correto, mas a divergência de cobertura precisa estar documentada para quem compara as duas rotas).

Condições de explorabilidade Não requer condição; so não se deve ler 'PASS' como prova de execução.

Impacto Leitura errada de vereditos; ruído em cross-checks entre implementações.

Correção sugerida Renomear/rotular o status (`CONFORMANCE\_PASS`) e listar na tabela de fronteira os limites próprios de cada host (`C0\_LIMIT\_\*`) ao lado dos do Stage0.

Critérios de aceite

- [] O veredito de `integrity` declara explicitamente que nenhuma execução ocorreu.
- [] `docs/V2\_LINVM\_AS\_COMPILER0\_NOZIG.rule1` traz a tabela de limites divergentes.
- [] Nenhum gate compara cobertura sem mencionar os caps de cada host.

4.6 C0 - ESCOPO NAO APLICAVEL / HIGIENE

BAIXA

F-09 - CI sem bloco `permissions:` e com ação de terceiro referida por tag mutável

Arquivo:linha .github/workflows/ci.yml:1-23 (sem `permissions:`), :27 (`goto-bus-stop/setup-zig@v2`), :88 (`upload-artifact@v4`)

Evidência

```
on:
  push: { branches: [master, main] }
  pull_request:
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: goto-bus-stop/setup-zig@v2 # tag mutável, não fixada em SHA
```

Por que é explorável Sem `permissions:` explícito, o `GITHUB\_TOKEN` herda a política padrão do repo (frequentemente escrita em contents/metadados); e a ação de terceiros é consumida por tag, que pode ser movida a monte. Num repositório cuja superfície de segurança é 'verifique os hashes você mesmo', a cadeia de CI é o elo mais fraco.

Condições de explorabilidade Requer que o padrão do repo conceda escrita ou que a tag upstream seja comprometida (ataque de supply chain clássico).

Impacto Superfície maior do que o necessário no runner; risco de troca do fornecedor da toolchain.

Correção sugerida Adicionar `permissions: { contents: read }` (e escrita so no job de release) e fixar `uses: ...@<sha>` com o número da tag em comentario.

Critérios de aceite

- [] `gh api repos/kbelludoo/lin-open/actions/permissions` (ou inspeção do YAML) mostra escopo minimo por job.
- [] Toda `uses:` de terceiro apontando para SHA de 40 hex.
- [] O job de release (quando existir) declara escrita apenas nele.

5. Recomendações priorizadas

P	Ação	Como	Achados
P1	Ancorar toda verificação criptografica em confiança externa ao documento	Roster/allowlist de authorities pinado (arquivo versionado + assinado por maintainers, fora do espaço do artefato), politica minima de quorum no verificador ($N \geq 3$, $M > 2N/3$), checagem de revogação/epoch. Elimina F-01 e F-02.	F-01, F-02, F-03, F-04, F-06
P1	Reclassificar/retirar o rótulo 'cryptographically verified' dos receipts e publicar um verificador com re-execução	Veredicto deve dizer `verification_level` do receipt; o verificador de nível 1 re-executa na VM (ou na LinVM `lin_c0`) e compara value+steps+sp_at_ret. Corrige F-03 e a leitura errada em F-11.	F-01, F-02, F-03, F-04, F-06
P1	Permissoes de arquivo de chave e recusa de defaults em rotas de segredo	`mode = 0o600` em todo createFile de segredo (8041, 9173), sem default `authority.key`/seed prefix, erro explicito quando o arquivo existente e legível por grupo/outros. Corrige F-04 e F-06.	F-01, F-02, F-03, F-04, F-06
P2	Gate obrigatório, assinado e em CI	`make gate` com `--roster` default, portar a checagem de assinatura para a rota python, aplicar o job `compiler0-no-zig` + um job `gate` na CI e exigir o check para merge. Corrige F-05.	F-05, F-07
P2	Nenhuma escrita/execucao com caminho implicito	Flags obrigatórias para saída, `O_NOFOLLOW`/stat antes de escrever, `--zig` explicito (ou PATH + hash esperado), remover a sonda `/home/k/`. Corrige F-07.	F-05, F-07
P3	Hardening de CI e higiene de docs	`permissions:` por job, ações fixadas por SHA, docs dizendo o que roda em CI hoje, remover/refazer a referencia a `nucleus.lock.json`. Corrige F-08 e F-09.	F-08, F-09, F-10, F-11
P3	Corrigir a alegação sobre o loader e deduplicar a busca de função	Textos e comentario de F-10; manter uma única implementação (ou justificar a copia como independencia de host).	F-08, F-09, F-10, F-11

Sequência sugerida: (i) P1 de ancoranca (F-01/F-02) primeiro, porque sem âncora qualquer outro controle e decorativo; (ii) P1 de rótulos/re-execucao do receipt (F-03) e permissões de chave (F-04/F-06), que são mudanças pequenas e fecham as superfícies externas; (iii) P2 no CI/gate e paths; (iv) P3 como higiene.

6. ISSUES PARA O GITHUB

Bloco por achado acionável, pronto para copiar e colar (também gravado em docs/security-audit/issues/). Os delimitadores --- ISSUE n --- / --- FIM ISSUE n --- existem só para marcar o início/fim do bloco e não fazem parte da issue. F-10 e F-11 foram agrupados numa única issue de correção de documentação/higiene para evitar spam.

```
--- ISSUE 1 --- arquivo: docs/security-audit/issues/01-verificadores-criptograficos-confiam-na-chave-pu.md
# [Seguranca] Verificadores criptograficos confiam na chave pública embutida no próprio documento verificado

**Labels sugeridas:** `security`, `security/critical`, `c1`
**Categoria da auditoria:** C1 isolamento/ancoragem
**Severidade:** CRITICA
**ID do achado:** F-01

## Problema e por que é explorável

A assinatura Ed25519 é válida contra a chave pública que o próprio documento declara.
Qualquer pessoa gera um par de chaves, assina as próprias alegações e obtém um documento que
passa em todas as rotas de verificação. Não há roster pinado, nenhuma âncora externa e
nenhuma checagem de revogação nessas rotas. O resultado impresso é 'ATTESTATION VALID' /
'PROVENANCE ... Verified authentic git lineage'.

## Evidência (arquivo:linha)

`compiler/lin.zig:8607-8621 (attest-verify); compiler/lin.zig:9806-9832 (verify-004); compiler/lin.zig:10392-10420 e 1
0623 (bundle-pack/bundle verify); compiler/lin.zig:11223 (ref externa)`
...
// attest-verify - compiler/lin.zig:8607-8621
// 7. Decode Key & Signature DIRECTLY from Document & Cryptographically Verify
if (doc.pubkey_hex.len != 64 or doc.signature_hex.len != 128) {
    return error.InvalidKeyOrSignatureLength;
}
var pubkey_raw: [32]u8 = undefined;
_ = try std.fmt.hexToBytes(&pubkey_raw, doc.pubkey_hex);
...
const pubkey = try std.crypto.sign.Ed25519.PublicKey.fromBytes(pubkey_raw);
try sig.verify(canonical_msg, pubkey); // <- a chave vem do documento assinado
...

## Condições de explorabilidade

Sem condição: qualquer `rule` produzido com `attest-issue --key <arquivo meu>` (ou com
chave própria + edição dos campos) é aceito. O `nversion`/`cert` seguem o mesmo padrão de
ancoragem.

## Impacto

Provas de execução/provenance deixam de ser provas. Um agente que queira publicar um
resultado adulterado pode emitir atestado 'valido' e o verificador de terceiros confirma.
Destroi a premissa central do projeto (receipt/attestation verificável).

## Sugestão de correção

Resolver a chave contra uma âncora fora do documento: roster/allowlist de authorities pinado
em arquivo versionado e assinado (ou policy de transparência tipo Sigstore), e recusar
quando `pubkey_hex` não pertence ao conjunto. Checar revogação/epoch na mesma rota. O roster
de notários deve chegar por canal fora de faixa (out-of-band), não do diretório do artefato.

## Critérios de aceite

- [ ] `attest-verify` de um documento assinado por chave fora do roster pinado sai BLOCKED com código nomeado.
- [ ] O roster usado na verificação não é lido do CWD nem do mesmo diretório do documento.
- [ ] Teste adversário: `attest-issue --key <chave nova>` + `attest-verify` => REJEITADO (hoje PASSA).
- [ ] Documentação deixa de dizer 'authenticated' sem âncora de confiança explícita.

---
_Gerado por `docs/security-audit/gen_report.py` (auditoria de 1 de setembro de 2026)._
--- FIM ISSUE 1 ---
```

```

--- ISSUE 2 --- arquivo: docs/security-audit/issues/02-quorum-de-testemunhas-e-auto-definido-roster-de-.md
# [Segurança] Quorum de testemunhas e auto-definido: roster de 1-de-1 com chave derivada de constante pública e aceito

**Labels sugeridas:** `security`, `security/critical`, `c2`
**Categoria da auditoria:** C2 confiança no lado errado
**Severidade:** CRITICA
**ID do achado:** F-02

## Problema e por que é explorável

O arquivo de roster contém simultaneamente as identidades (pubkeys), as assinaturas e o
limiar de quorum. Como `quorum_threshold=1` com `N=1` passa na checagem de maioria e as
chaves podem ser regeneradas de um prefixo público, o verificador aceita o checkpoint que um
atacante quiser. PoC executado neste ambiente: `lin notary-sign -o forged.rule1 --witnesses
1 --threshold 1 --tree-size 999999 --epoch 7 --state-root sha256:aaaa...` seguido de `lin
notary-verify` => `[QUORUM SATISFIED]` e `TRANSPARENCY CHECKPOINT VERIFIED`, com receipt
escrito.

## Evidência (arquivo:linha)

`compiler/lin.zig:7284-7338 (parse + politica de quorum), 7363-7380 (verifyOne), 12455-12615 (notary-sign, seed default
t), 12643-12700 (notary-verify le witness_roster.rule1 do CWD)`

...
// compiler/lin.zig:7284-7292 - tudo vem do MESMO arquivo que se verifica
pub fn parse(alloc, bytes) !Parsed {
    if (!std.mem.startsWith(u8, bytes, "@RULE1:LIN_WITNESS_ROSTER:1.0.0")) ...
        .quorum_threshold = findIntField(bytes, "quorum_threshold") orelse ...,
}
// compiler/lin.zig:7337-7341 - politica aceita 1-de-1
if (parsed.head.quorum_threshold == 0) return error.QuorumThresholdZero;
if (parsed.head.quorum_threshold > n) return error.QuorumThresholdExceedsRoster;
if (parsed.head.quorum_threshold * 2 <= n) return error.QuorumThresholdNotMajority;
// compiler/lin.zig:12457 - material de chave publico por default
var seed_prefix: []const u8 = "lin-transparency-selftest";
...

## Condições de explorabilidade

Explorável por qualquer pessoa que consiga colocar `witness_roster.rule1` no diretório de
execução (o caminho é fixo e relativo ao CWD: compiler/lin.zig:12643) - por exemplo no mesmo
tree de um PR. `notary-verify <arquivo>` ignora o argumento posicional e lê o default, então
nem o operador percebe a troca.

## Impacto

Log de transparência forjável de ponta a ponta: árvore, epoch e state root arbitrários
`notarizados`. Todo consumidor do receipt `notarization_status=QUORUM_VERIFIED_FROM_ROSTER`
é enganado.

## Sugestão de correção

Pin set de pubkeys de notários fora do espaço do artefato (arquivo assinado / config de
deploy), exigir `N >= 3` e `M > 2N/3`, recusar rosters com testemunhas novas e exigir que
`--roster` seja obrigatório (sem modo `sem testemunhas`). Rejeitar argumentos posicionais
desconhecidos em rotas de verificação.

## Critérios de aceite

- [ ] `notary-verify` falha se `quorum_threshold`/`N` não baterem com a política pinada (N>=3).
- [ ] Um roster com testemunha desconhecida => `QUORUM NOT SATISFIED` + exit != 0.
- [ ] `notary-verify <arquivo>` usa o arquivo informado ou erro de uso (não lê o CWD as escondidas).
- [ ] Nenhum default de seed prefix em rota que gera chaves; `--seed-prefix` passa a ser obrigatório.

---
_Gerado por `docs/security-audit/gen_report.py` (auditoria de 1 de setembro de 2026)._
--- FIM ISSUE 2 ---

```



```
--- ISSUE 3 --- arquivo: docs/security-audit/issues/03-receipts-cryptographically-verified-sao- apenas-a.md
# [Segurança] Receipts 'cryptographically verified' são apenas autocoerência de hash - e a decisão de 'verificado' acontece no navegador
```

```
**Labels sugeridas:** `security`, `security/high`, `c2`
**Categoria da auditoria:** C2 confiança no lado errado
**Severidade:** ALTA
**ID do achado:** F-03
```

Problema e por que é explorável

input, output, steps e sp_at_ret vem do receipt; o verificador só recomputa SHA-256(leaf) e compara. Não há chave, assinatura, nem re-execução. PoC neste ambiente: receipt forjado com `output=12345` (a função é `x\*x`, entrada 9, saída correta 81) => `node verify\_receipt.js` imprime PASS e `lin receipt-verify --receipt` imprime 'PASS: Compute Receipt ... cryptographically verified!'.

Evidência (arquivo:linha)

```
`compiler/lin.zig (receipt-verify), benchmarks/verify_receipt.js:1-20 e 96-131, benchmarks/verify_receipt.html:19-27 e 100-140, tool/lin_c_receipt.c (so emissor, sem verificador na porta C11)`
```

```
...
```

```
// benchmarks/verify_receipt.html:100-140 (e o mesmo em verify_receipt.js)
const computed = await sha256(leaf); // leaf = campos do proprio receipt
let ok = computed === claimed;
if (source) { ... } // checagem do artifact e OPCIONAL
out.textContent = ... + (ok ? "[PASS] Receipt validated by an independent open-source verifier"
: "[FAIL] Receipt forged or tampered!");
...
```

Condições de explorabilidade

Sem condição: basta saber a fórmula aberta do formato, documentada na própria página ('Open format: merkle_root = SHA-256(...)').

Impacto

Falsa sensação de prova. Um verifier terceirizado (a página é distribuída como 'independent') aprova qualquer resultado. O rótulo 'cryptographically verified' é overclaim: o receipt honestamente se declara `verification\_level=0` / `INTEGRITY\_RECEIPT\_LOCAL`, mas a saída do verificador não preserva esse nível.

Sugestão de correção

Renomear o veredicto para o que ele é ('autoconsistente / nível 0') e imprimir o nível declarado no receipt; para virar prova, vincular o receipt à execução: re-executar na LinVM (`lin\_c0 vm` / `lin vm`) e comparar value+steps+sp_at_ret, ou então exigir assinatura de um emissor pinado (ver F-01). Manter a checagem do `artifact` obrigatória quando houver fonte.

Critérios de aceite

- [] A página/CLI exibem `verification\_level` e a frase do veredicto não usa 'cryptographic' para nível 0.
- [] Receipt com output alterado e `--source` informado => FAIL (hoje o source check é opcional).
- [] Um verificador de nível 1+ (re-execução na VM) existe e falha para receipt forjado.
- [] Nenhum caminho de CI apresenta nível 0 como prova de execução.

```
---
```

```
_Gerado por `docs/security-audit/gen_report.py` (auditoria de 1 de setembro de 2026)._
```

```
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 --- arquivo: docs/security-audit/issues/04-chave-privada-de-autoridade-gerada-com-permissao.md
# [Segurança] Chave privada de autoridade gerada com permissão 0644 e nome de arquivo default no CWD

**Labels sugeridas:** `security`, `security/high`, `c4`
**Categoria da auditoria:** C4 chaves/secrets
**Severidade:** ALTA
**ID do achado:** F-04

## Problema e por que é explorável

`createFile(path, .{})` usa 0o666 & ~umask => 0644 com a umask padrão: o seed privado de 256
bits fica legível por qualquer usuário da máquina e por qualquer job/artefato que cole o
diretório (upload-artifact, `git add -A`, tarball de debug). O próprio repo já sabe fazer
certo (`gate-keygen` usa 0o600, medido: `lin_gate_key.seed` saiu 600), então é uma
inconsistência na rota de attest - não falta mecânica, falta aplicá-la ali.

## Evidência (arquivo:linha)

`compiler/lin.zig:7643` e `8719` (default "authority.key"), `8038-8044` e `9170-9176` (geração + createFile sem modo), contra
ste correto em `compiler/lin.zig:15761` (gate-keygen usa `.mode = 0o600`)

```
// compiler/lin.zig:7643 - default relativo ao CWD
var key_path: []const u8 = "authority.key";
// compiler/lin.zig:8038-8044 - gera e escreve sem fixar modo
std.crypto.random.bytes(&authority_seed);
_ = try std.fmt.bufPrint(&key_hex_buf, "{s}", .{std.fmt.fmtSliceHexLower(&authority_seed)});
const kfile = try std.fs.cwd().createFile(key_path, .{});
try kfile.writeAll(&key_hex_buf);
// compiler/lin.zig:15761 - a outra rota FAZ o certo
const kf = try std.fs.cwd().createFile(key_out, .{ .truncate = true, .mode = 0o600 });
```

## Condições de explorabilidade

Local: qualquer processo do mesmo host lê o seed. Em CI: um `upload-artifact` com caminho
amplo (`**`) publica o arquivo; o .gitignore do repo e allowlist, então `authority.key` não
seria commitado - mas um `git add -f` ou um bundle da árvore expõe.

## Impacto

Roubo da identidade de assinatura de atestados: o atacante passa a emitir atestados válidos
(com F-01, sem nem precisar do verificador da vítima).

## Sugestão de correção

Usar `.mode = 0o600` em todo createFile de material sensível (8041, 9173), recusar default
`authority.key` se o arquivo já existir e não for modo 0600 (validação de startup), e
imprimir aviso quando o caminho estiver dentro de uma árvore de trabalho.

## Critérios de aceite

- [ ] Após `attest-issue`, `stat -c %a authority.key` == 600 (hJe não roda porque o arquivo existe antes de mim).
- [ ] Rodar com `--key <arquivo existente 0644>` => erro nomeado 'key file is group/other readable'.
- [ ] grep por `createFile` em rotas de chave não deixa nenhum call sem `.mode`.
- [ ] Teste de CI: `upload-artifact` com `path: .` falha se houver chave no tree.

---
_Gerado por `docs/security-audit/gen_report.py` (auditoria de 1 de setembro de 2026)._
--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 --- arquivo: docs/security-audit/issues/05-o-lin-gate-e-auto-atestavel-por-default-sem-rost.md
# [Segurança] O LIN Gate e auto-atestavel por default: sem roster, quem adultera o compilador também limpa o gate

**Labels sugeridas:** `security`, `security/medium`, `c1`
**Categoria da auditoria:** CI isolamento/ancoragem
**Severidade:** MEDIA
**ID do achado:** F-05

## Problema e por que é explorável

O gate detecta adulteração, mas não prova quem a re-atestou: `python3
test/verify_gate_manifest.py --attest` regravava a raiz e o `gate-check` volta a dizer OPEN.
PoC neste ambiente: acrescentei uma linha em `compiler/lin.zig` => `error: GateBlocked` e
`GATE BLOCKED` no python; rodei o `--attest` => `GATE OPEN` - a raiz do disco coincide com a
atestada. O modo assinado existe e é fail-closed (`gate-check --roster` sobre manifesto não
assinado => `GATE BLOCKED - attestation signature not verified`, medido), porém é opcional,
o roster é lido de caminho arbitrário (logo editável no mesmo PR) e a porta sem-Zig nem
implementa assinatura.

## Evidência (arquivo:linha)

`Makefile:170-186 (gate/gate-attest, GATE_ROSTER opcional), test/verify_gate_manifest.py:137-142 e 164-165 (sem suport
e a roster/assinatura), compiler/lin.zig:15678-15692 (claim 'a gate that blocks a pull request - AI-authored or not'),
.github/workflows/ci.yml (nenhum job chama o gate)`

...
# Makefile:176-186
gate: build-cpu
ifeq ($(GATE_ROSTER),)
@$(BIN) gate-check
else
@$(BIN) gate-check --roster $(GATE_ROSTER)
endif
# test/verify_gate_manifest.py:165
print(" manifesto escrito: %s (attested_by=%s - NAO assinado)" % (manifest, args.attested_by))
`

## Condições de explorabilidade

Explorabilidade real = 'quem pode abrir PR contra a master': não precisa de segredo, apenas
do direito de editar o arquivo de manifesto (o mesmo direito já dá para editar o
compilador). A mitigação depende de branch protection + revisão humana, não do gate.

## Impacto

Controle de integridade de supply chain descrito como bloqueante e, na prática, advisory; em
particular o job `compiler0-no-zig` proposto (docs/CI_COMPILER0_NO_ZIG_JOB_PATCH.rule1) usa
apenas a porta python não-assinada.

## Sugestão de correção

Tornar `--roster` obrigatório no `make gate` (com roster pinado fora da árvore do PR ou
assinado por maintainers), portar a checagem de assinatura para
`test/verify_gate_manifest.py`, e registrar o gate num job de CI (bloqueando merge).

## Critérios de aceite

- [ ] `make gate` sem GATE_ROSTER falha (não é mais modo padrão).
- [ ] `python3 test/verify_gate_manifest.py --roster <R>` reproduz o veredito assinado do Zig.
- [ ] O job de CI executa gate + roster e o merge exige check verde.
- [ ] Adulterar compiler/lin.zig + reatestar sem a chave do maintainer => CI vermelho (prova de PoC).

---
_Gerado por `docs/security-audit/gen_report.py` (auditoria de 1 de setembro de 2026)._
--- FIM ISSUE 5 ---
```

```

--- ISSUE 6 --- arquivo: docs/security-audit/issues/06-material-de-chave-derivado-de-constante-publica-.md
# [Segurança] Material de chave derivado de constante pública e sem validação de startup (defaults que viram segredo real)

```

```

**Labels sugeridas:** `security`, `security/medium`, `c4`
**Categoria da auditoria:** C4 chaves/segregos
**Severidade:** MEDIA
**ID do achado:** F-06

```

Problema e por que é explorável

O comentário no código é honesto ('NOT a root of trust'), mas o default está vivo em rota de linha de comando que gera chaves e pública roster; o ID da testemunha ainda carrega o prefixo, facilitando reuso. Não há flag que recuse produzir um roster 'de selftest' num contexto de release, nem checagem de que o prefixo veio de fora.

Evidência (arquivo:linha)

```

`compiler/lin.zig:12457 (seed_prefix default), 7383-7395 (deriveKeypair), 12591-12601 (id da testemunha contém o prefixo), 12613-12615 (apenas um WARNING impresso)`

```

```

...
// compiler/lin.zig:12457 + 7386-7395
var seed_prefix: []const u8 = "lin-transparency-selftest";
pub fn deriveKeypair(seed_prefix: []const u8, index: usize) !KeyPair {
    h.update("lin:transparency:witness-key:v1:"); h.update(seed_prefix); ...
    return Ed25519.KeyPair.create(seed); // determinístico a partir do prefixo
}
...

```

Condições de explorabilidade

Basta rodar `lin notary-sign` sem `--seed-prefix` (default) e usar o resultado como prova - e o que F-02 demonstra.

Impacto

Chaves de testemunha colecionadas por qualquer pessoa => quorum sem valor (multiplica F-02).

Sugestão de correção

Exigir `--seed-prefix`/`--key` explícito (sem default) e, em modo release, importar pubkeys de notários de arquivo assinado; marcar o roster gerado com `selftest=true` e fazer o verificador recusar esse flag por default.

Critérios de aceite

- [] `notary-sign` sem seed prefix => erro de uso (hoje assume default).
- [] Roster autoassinado carrega `selftest=true` e `notary-verify` o recusa a menos de `--allow-selftest`.
- [] README/docs de deploy não mostram mais o comando sem `--seed-prefix`.

```

---
_Gerado por `docs/security-audit/gen_report.py` (auditoria de 1 de setembro de 2026)._
--- FIM ISSUE 6 ---

```

```
--- ISSUE 7 --- arquivo: docs/security-audit/issues/07-escritas-em-caminho-fixe-dentro-da-arvore-de-tra.md
# [Seguranca] Escritas em caminho fixo dentro da arvore de trabalho e execução de binario escolhido por env/sonda de filesystem
```

```
**Labels sugeridas:** `security`, `security/medium`, `c3`
**Categoria da auditoria:** C3 referencia nao validada
**Severidade:** MEDIA
**ID do achado:** F-07
```

Problema e por que é explorável

Tres problemas no mesmo ponto: (i) `rebuild` sobrescreve `src/lin.zig` sem checagem de symlink nem confirmação - arquivo pre-criado como link vira escrita fora da arvore; (ii) `pick\_target` faz o resultado da verificação depender de um caminho absoluto de um desenvolvedor (`/home/k/...`), entao `zig` vs `js` muda de maquina para maquina num projeto que vende reprodutibilidade bit a bit; (iii) o binario do compilador usado para compilar e EXECUTAR o código gerado vem de `LIN\_ZIG`, sem validação alguma.

Evidência (arquivo:linha)

`compiler/lin.zig:7549 (lin rebuild escreve src/lin.zig), 7129-7137 (pick\_target escreve .lin\_ast\_check.zig no CWD e p refere /home/k/.local/bin/zig se existir), 7018-7027 (LIN\_ZIG decide o `zig run` do codigo gerado por `lin test`)

```
```
```

```
// compiler/lin.zig:7549 - write em caminho fixo do source tree
const tf = try std.fs.cwd().createFile("src/lin.zig", .{});
// compiler/lin.zig:7129-7137 - alvo escolhido por presenca de arquivo no HOME de um dev
const tmp_file = ".lin_ast_check.zig";
if (std.fs.cwd().createFile(tmp_file, .{})) |f| { ... }
if (std.fs.openFileAbsolute("/home/k/.local/bin/zig", .{})) |zf| { zb = "/home/k/.local/bin/zig"; }
// compiler/lin.zig:7018-7027
var zb: []const u8 = "zig";
if (std.process.getEnvVarOwned(LIA_ALLOC, "LIN_ZIG")) |p| { zb = p; }
return std.process.Child.run(.{ .argv = &.{ zb, "run", "-0", "ReleaseFast", tmp_path }, ... });
```
```

Condições de explorabilidade

Requer a vitima rodar `lin rebuild`/`lin test` num diretorio controlado por terceiro (compartilhado, CI com checkout de PR, workspace de review) ou com ambiente em que o atacante seta variaveis.

Impacto

Escrita arbitraria/quebra de arquivo-fonte; resultado de verificação não reproduzível; execução de binario arbitrario com o código gerado a partir do input (RCE em fluxo de review/CI).

Sugestão de correção

Escrever em caminho explicito via flag (sem default no source tree), abrir com `O\_NOFOLLOW`/verificar symlink, e exigir `--zig <caminho>` (ou resolver so pelo PATH, com hash esperado) para o compilador externo; substituir a sonda `/home/k/...` por config explicita.

Critérios de aceite

- [] `lin rebuild` recusa escrever fora do diretorio informado e não toca em caminho fixo.
- [] Nenhum caminho absoluto de usuario aparece no código-fonte (grep `/home/` em compiler/*.zig => 0).
- [] `lin test` com `LIN\_ZIG=/bin/false` falha com erro nomeado, sem executar o que foi gerado.
- [] O alvo escolhido (zig/js) sai impresso com a justificativa e e identico em duas maquinas limpas.

```
---
```

Gerado por `docs/security-audit/gen\_report.py` (auditoria de 1 de setembro de 2026).
 --- FIM ISSUE 7 ---

```

--- ISSUE 8 --- arquivo: docs/security-audit/issues/08-documentacao-afirma-gates-publicacao-em-ci-que-o.md
# [Seguranca] Documentação afirma gates/publicacao em CI que o workflow não contem (falsa garantia)

**Labels sugeridas:** `security`, `security/low`, `c5`
**Categoria da auditoria:** C5 tratamento de input
**Severidade:** BAIXA
**ID do achado:** F-08

## Problema e por que é explorável

O texto diz que a CI pública assets verificaveis; o workflow em arvore não tem o job (o
patch esta em `docs/CI_*_PATCH.rulel`, aguardando aplicação por falta de escopo
`workflows`). Quem seguir o doc para 'baixar e verificar' não encontra o asset nem a
verificação. O arquivo `nucleus.lock.json`, citado como trava de integridade do verificador,
não existe.

## Evidência (arquivo:linha)

`.github/workflows/ci.yml` (92 linhas; nenhum job 'release' nem 'gate' nem 'compiler0-no-zig'), docs/LIN_SEM_ZIG_CAMINH
OS.rulel:24-31, docs/CI_RELEASE_E_V1_GATES_PATCH.rulel, docs/CI_COMPILER0_NO_ZIG_JOB_PATCH.rulel, HISTORY.rulel:78 (nu
cleus.lock.json inexistente)`

...
docs/LIN_SEM_ZIG_CAMINHOS.rulel: "Este PR fecha essa divergencia: a CI agora sobe os
artifacts lin_native-cpu/lin_native-gpu em toda execucao e publica os dois como assets de
Release ao empurrar tag `v*` (job `release`, --verify-tag)."
$ grep -c "release" .github/workflows/ci.yml -> 0
$ ls nucleus.lock.json -> No such file or directory
...

## Condições de explorabilidade

Sem condição tecnica; e leitura de documentação. Materializa-se quando alguém confia na
promessa em vez do manifesto.

## Achados agrupados neste ticket

- **F-09 (baixa)** - CI sem bloco `permissions:` e com ação de terceiro referida por tag mutavel
  - Evidencia: `.github/workflows/ci.yml:1-23` (sem `permissions:`), :27 (`goto-bus-stop/setup-zig@v2`), :88 (`upload-a
rtifact@v4`)
  - Aceite: `gh api repos/kbelludoo/lin-open/actions/permissions` (ou inspeção do YAML) mostra escopo minimo por job.;
Toda `uses:` de terceiro apontando para SHA de 40 hex.; O job de release (quando existir) declara escrita apenas nele
.
- **F-11 (informativa)** - Verificação de paridade/reportes com `steps=0` e status PASS; limites divergentes entre hos
ts
  - Evidencia: `compiler/lin.zig` (integrity/certificate), transpile/c/tool/lin_c0_front.c (C0_LIMIT_*), SECURITY_AUDIT
.md (prior art)`
  - Aceite: O veredito de `integrity` declara explicitamente que nenhuma execução ocorreu.; `docs/V2_LINVM_AS_COMPILER
0_NOZIG.rulel` traz a tabela de limites divergentes.; Nenhum gate compara cobertura sem mencionar os caps de cada host
.

## Impacto

Expectativa de seguranga falsa sobre a cadeia de suprimento do binario; usuarios instalam
asset inexistente/nao-verificado.

## Sugestão de correção

Deixar o status explicito nos docs ('patch pronto, NAO aplicado' já esta em
LIN_SEM_ZIG_CAMINHOS:linhas de faixa 1, mas a frase do release não) ou aplicar os jobs;
remover a referencia a `nucleus.lock.json` ou reintroduzi-la.

## Critérios de aceite

- [ ] Nenhum documento afirma que a CI pública assets enquanto o job não existir.
- [ ] Referencias a arquivos de trava apontam para arquivos presentes no tree.
- [ ] README/docs listam quais gates rodam em CI hoje (nenhum) e quais rodam so locais.

---
_Gerado por `docs/security-audit/gen_report.py` (auditoria de 1 de setembro de 2026)._
--- FIM ISSUE 8 ---

```

```

--- ISSUE 9 --- arquivo: docs/security-audit/issues/09-alegacao-tecnica-falsa-sobre-o-loader-de-confian.md
# [Seguranca] Alegação técnica falsa sobre o loader de confiança, duplicação da mesma lógica e comentario depreciativo
no documento de fronteira

**Labels sugeridas:** `security`, `security/info`, `c3`
**Categoria da auditoria:** C3 referencia nao validada
**Severidade:** INFORMATIVA
**ID do achado:** F-10

## Problema e por que é explorável

O `strlen` do loader é aplicado no nome solicitado (argv), nunca na pool; a comparação já
usa `name_len[i]`. Medido: `lin_bcl_run` e `lin_c0_run` resolvem nomes em qualquer posição
(`lex_gate`, `lex_scan_embedded`, `lex_count_embedded` = 1 / 1371904279658369433 / 20) e
recusam nome inexistente. O par de nomes com prefixo comum (`gate`, `gate_x`) resolve
corretamente pelas duas rotas. Logo: não ha vulnerabilidade, mas a alegação publicada (doc
$7, HISTORY e comentario) afirma que o TCB tem um furo e duplica a função do loader 'por
segurança'.

## Evidência (arquivo:linha)

`transpile/c/tool/lin_c0.c:156-165, docs/V2_LINVM_AS_COMPILER0_NOZIG.rule1 (secao 7), compiler0_manifest.rule1, HISTOR
Y.rule1 (entrada C0-no-Zig cont.)`

...
/* transpile/c/tool/lin_c0.c:156-165
 * Busca exata pelo nome usando name_len[] do loader: o pool LINBCL nao e
 * NUL-terminado, entao `strlen` (como faz lin_bcl_find_fn) so e seguro no
 * ultimo nome da secao. Aqui e comparacao por comprimento - correta sempre. */
static int bcl_find_fn_strict(const LinBclVm *m, const char *name) { ... }
// transpile/c/lin_c/lin_linbcl.c:195-202 (o loader JA compara por comprimento)
int lin_bcl_find_fn(const LinBclVm *m, const char *name) {
    size_t want = strlen(name);
    for (size_t i = 0; i < m->mod.fns_len; i++)
        if ((size_t)m->name_len[i] == want && memcmp(m->mod.fns[i].name, name, want) == 0)
            return (int)i;
    return -1; }
...

## Condições de explorabilidade

Não explorável. O custo e de confiança/revisao: quem audita confia menos no loader do que
deveria e mantem dois codigos identicos.

## Achados agrupados neste ticket

- **F-11 (informativa)** - Verificação de paridade/reportes com `steps=0` e status PASS; limites divergentes entre hos
ts
  - Evidencia: `compiler/lin.zig (integrity/certificate), transpile/c/tool/lin_c0_front.c (C0_LIMIT_*), SECURITY_AUDIT
.md (prior art)`
  - Aceite: O veredito de `integrity` declara explicitamente que nenhuma execução ocorreu.; `docs/V2_LINVM_AS_COMPILER
0_NOZIG.rule1` traz a tabela de limites divergentes.; Nenhum gate compara cobertura sem mencionar os caps de cada host
.

## Impacto

Documentação de fronteira incorreta + lógica redundante a `tool/` (manutencao).

## Sugestão de correção

Corrigir comentario e docs (remover a alegação), e ou chamar `lin_bcl_find_fn` diretamente
ou justificar a copia com o motivo real (independencia de host), não com um defeito
inexistente.

## Critérios de aceite

- [ ] grep 'so e seguro no último nome' não retorna nada no repo.
- [ ] Seção 7 do doc descreve `bcl_find_fn_strict` como comparação por comprimento equivalente a do loader.
- [ ] Um teste cobre resolução de nomes em primeira/meia/ultima posição (rota fonte e rota imagem).

---
_Gerado por `docs/security-audit/gen_report.py` (auditoria de 1 de setembro de 2026)._
--- FIM ISSUE 9 ---

```

Nenhuma das issues foi aberta no GitHub por esta auditoria: o objetivo era produzir o material revisavel. Para abrir tudo de uma vez: for f in docs/security-audit/issues/*.md; do gh issue create --title "\${head -1 \$f | sed 's/^# //'}" --body-file \$f; done